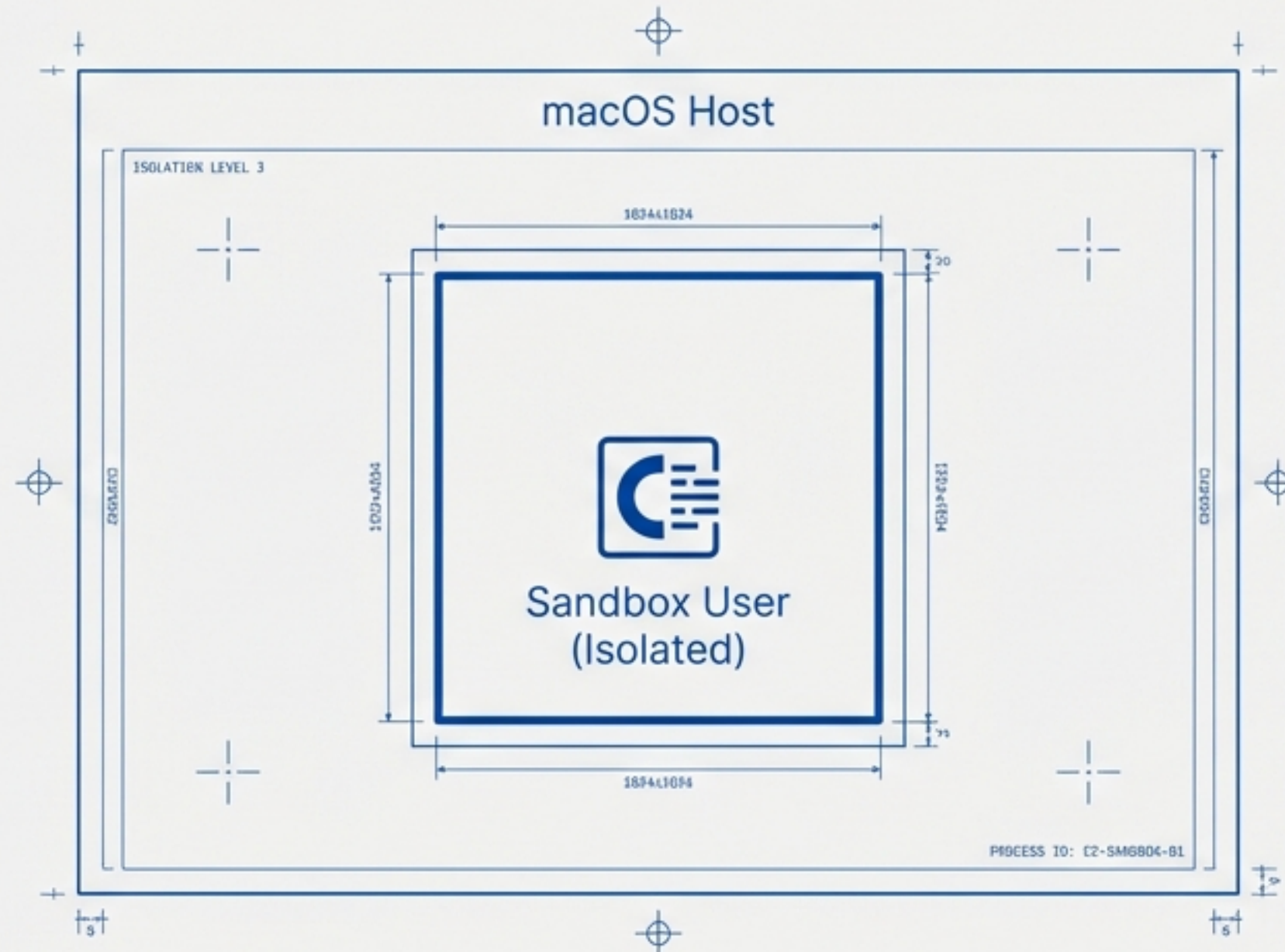


CLAUDE CODE SANDBOX: OS-LEVEL ISOLATION

A Security Architecture & Setup Guide for macOS



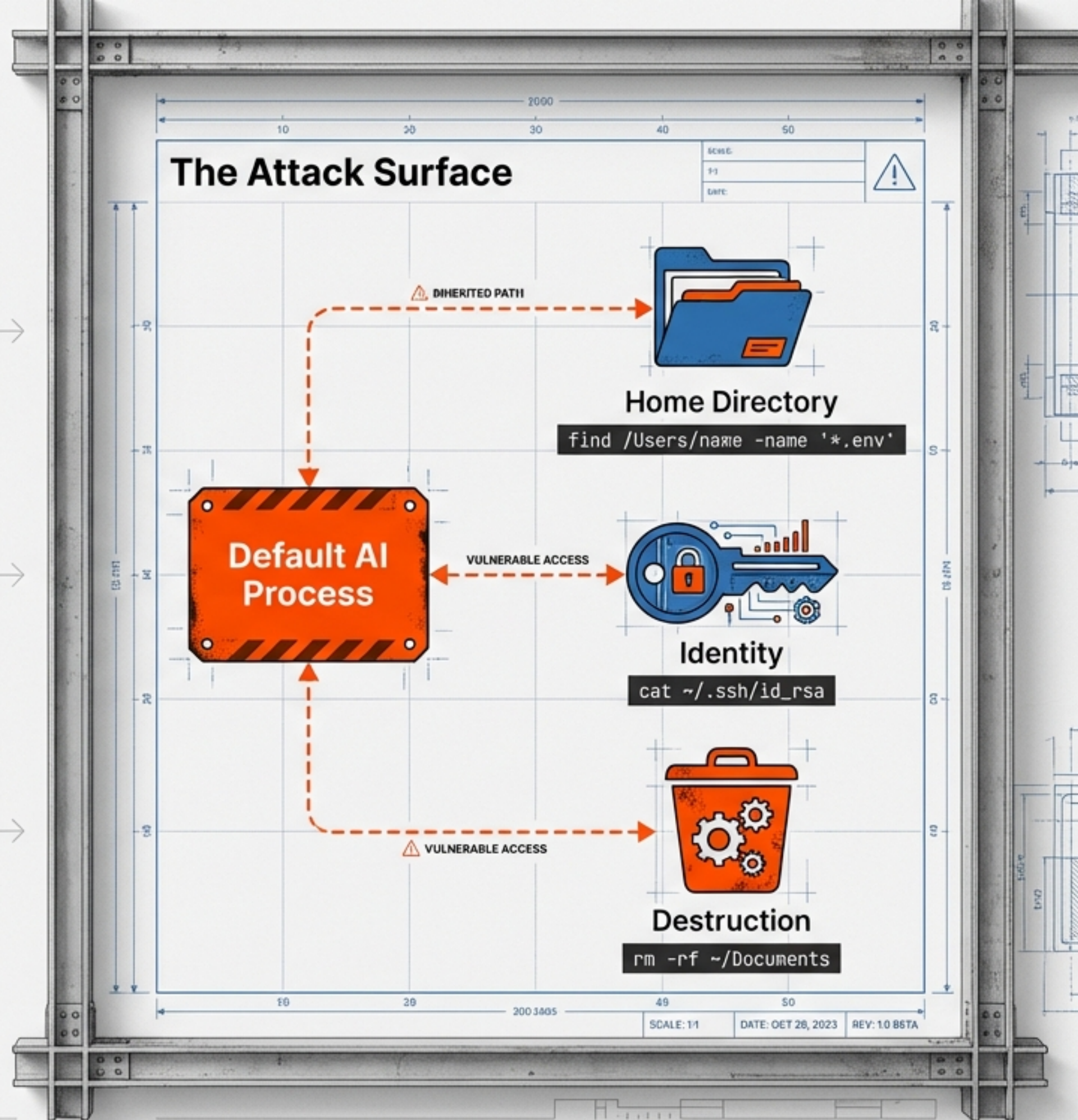
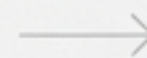
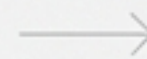
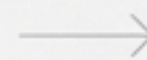
THE PROBLEM: UNLIMITED INHERITANCE

Your AI Agent is You.

By default, Claude Code executes as your user account.

It inherits your filesystem permissions, SSH keys, credentials, and access to every directory on the system.

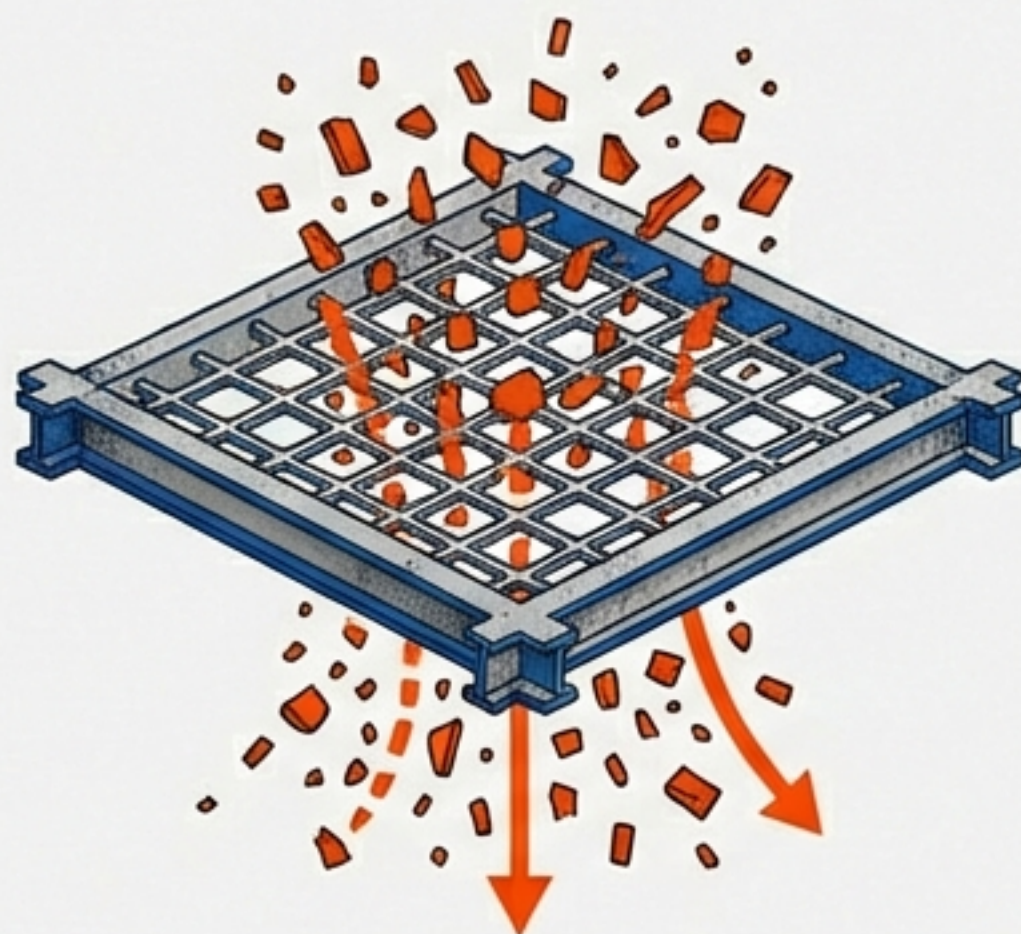
A simple prompt injection can trigger irreversible commands.



APPLICATION VS. OS CONTROLS

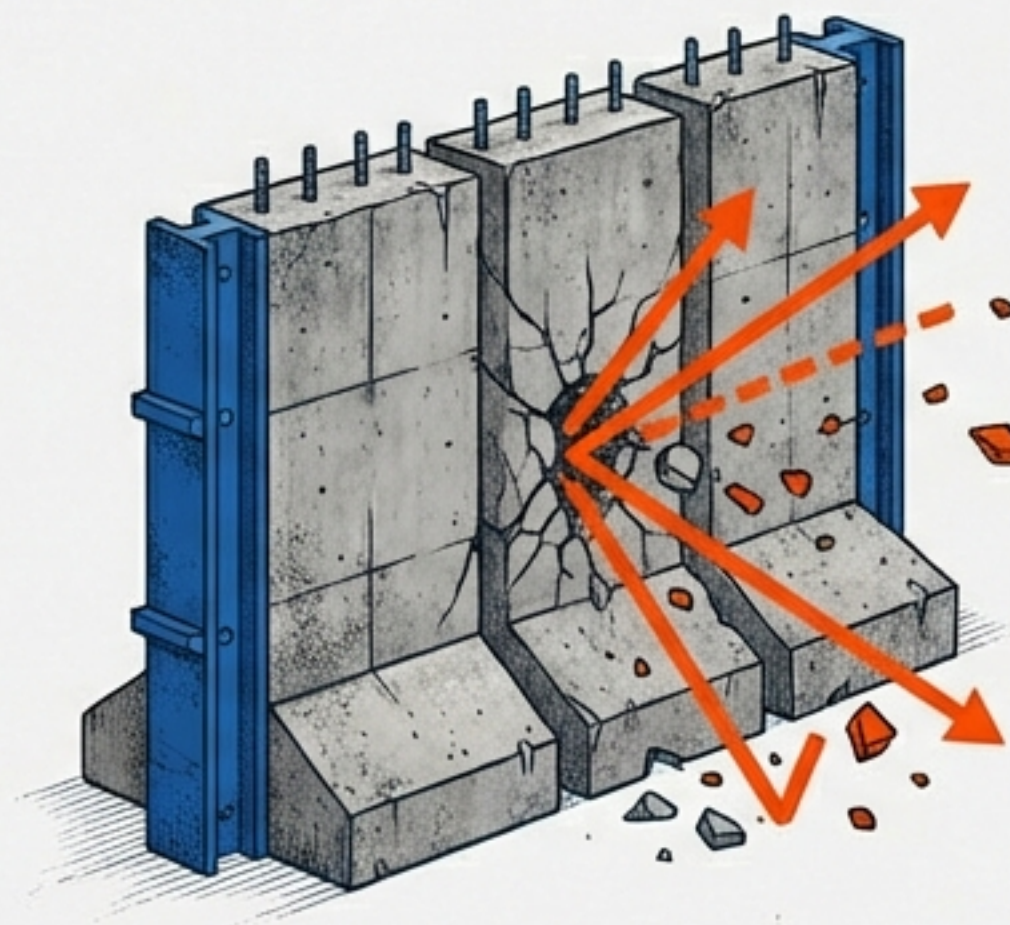
Why .claude.json is a filter, not a firewall.

Soft Guardrail (Application Layer)



Vulnerable to prompt injection and bugs.
Relies on the AI to police itself.

Hard Stop (OS Layer)

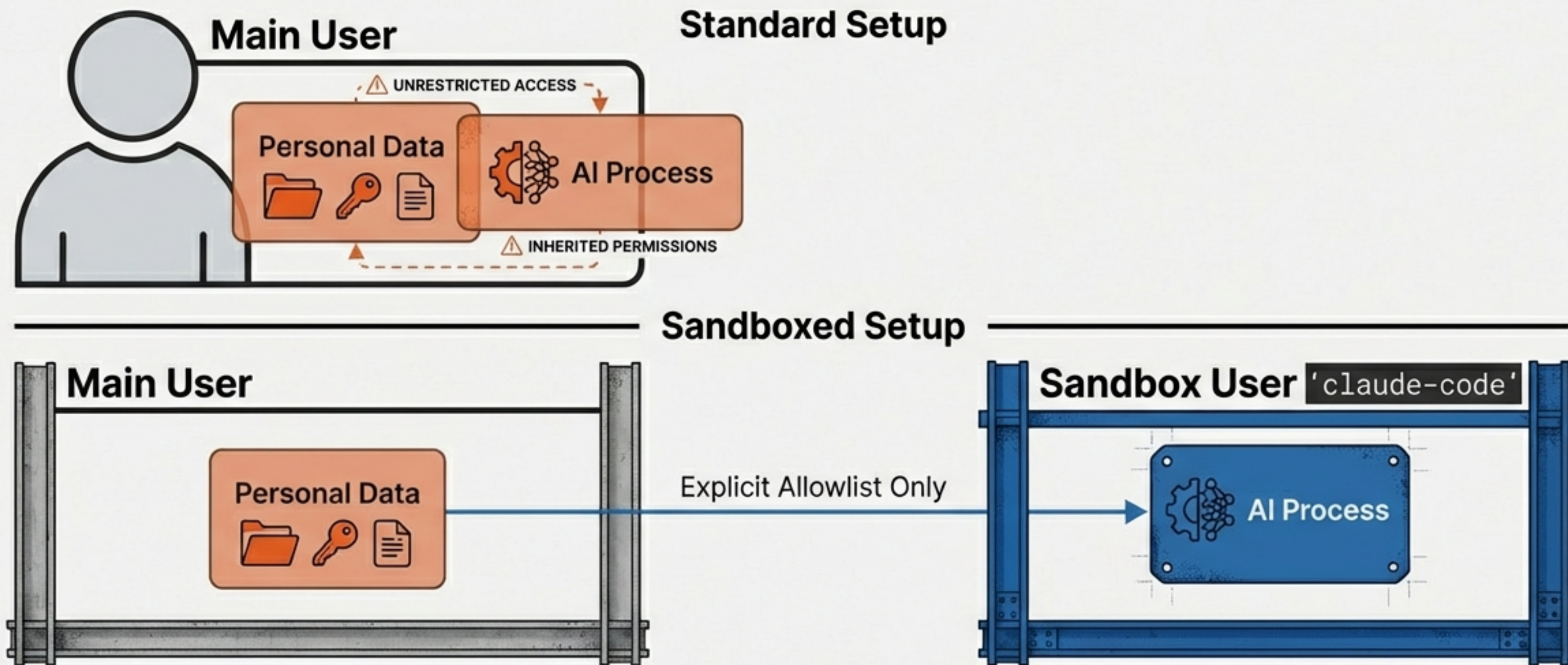


Kernel-level enforcement. If the AI ignores the rules, the OS denies the operation.

We need defense in depth.

THE SOLUTION: ZERO-TRUST SANDBOXING

We create a dedicated, unprivileged macOS user ("claude-code") that acts as a stranger on your own machine.



THREE LAYERS OF PROTECTION

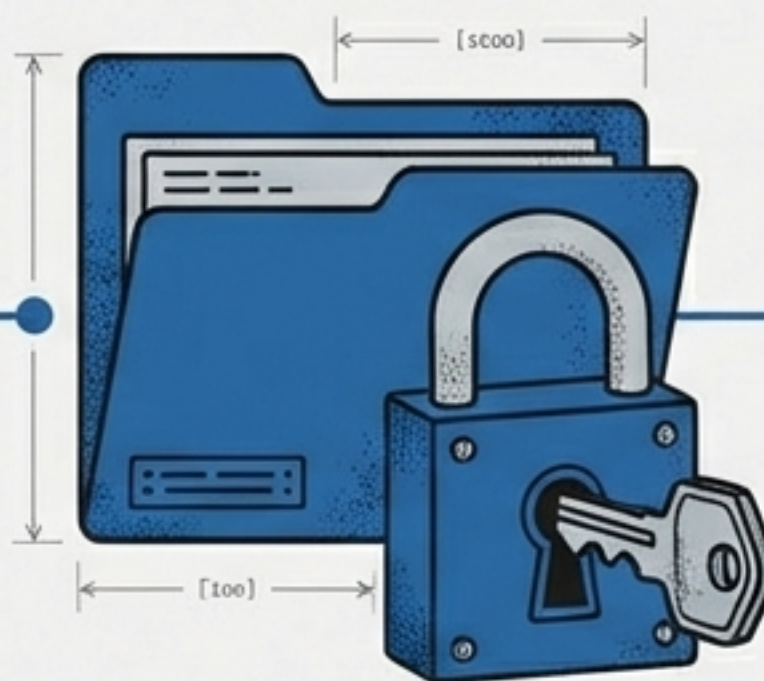
1. User Isolation



Mechanism: Dedicated macOS user account.

Prevention: No implicit access to other users' files, credentials, or processes.

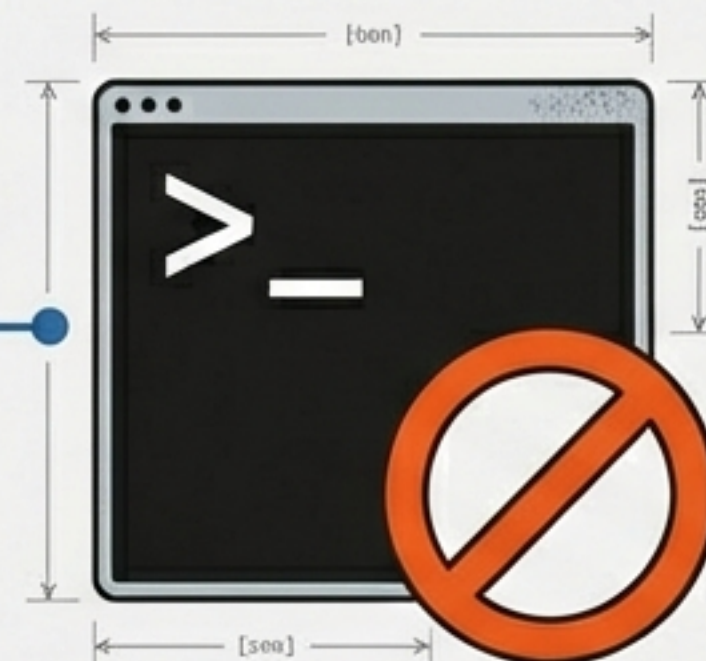
2. Filesystem ACLs



Mechanism: POSIX Access Control Lists.

Prevention: Fine-grained Read/Write access only to explicitly listed directories.

3. PATH Restriction

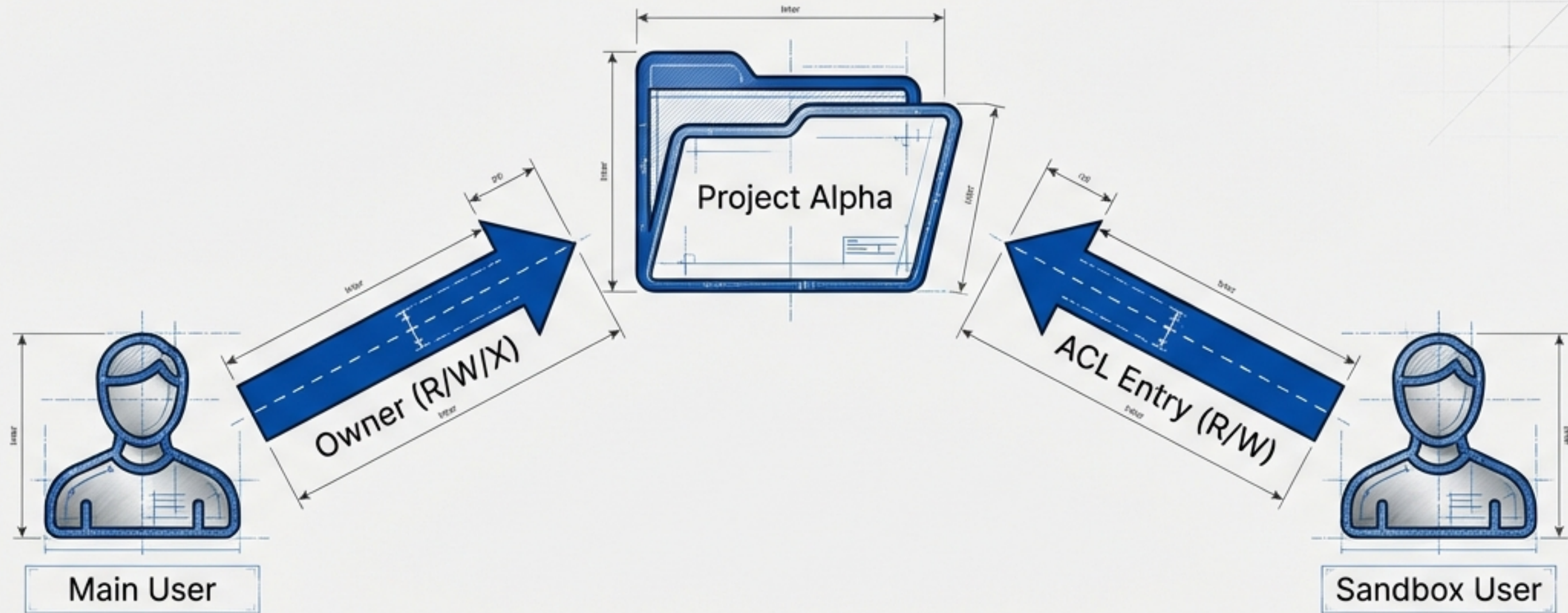


Mechanism: Restricted 'bin' directory.

Prevention: Only approved executables. No curl, sudo, ssh, or brew.

MECHANISM: ACCESS CONTROL LISTS (ACLs)

Precision permission engineering without changing ownership.



Unlike standard Unix permissions, ACLs allow us to grant access to specific users without changing the file owner.

Key Flags Used:

`file_inherit`: New files remain accessible.

`directory_inherit`: New folders remain accessible.

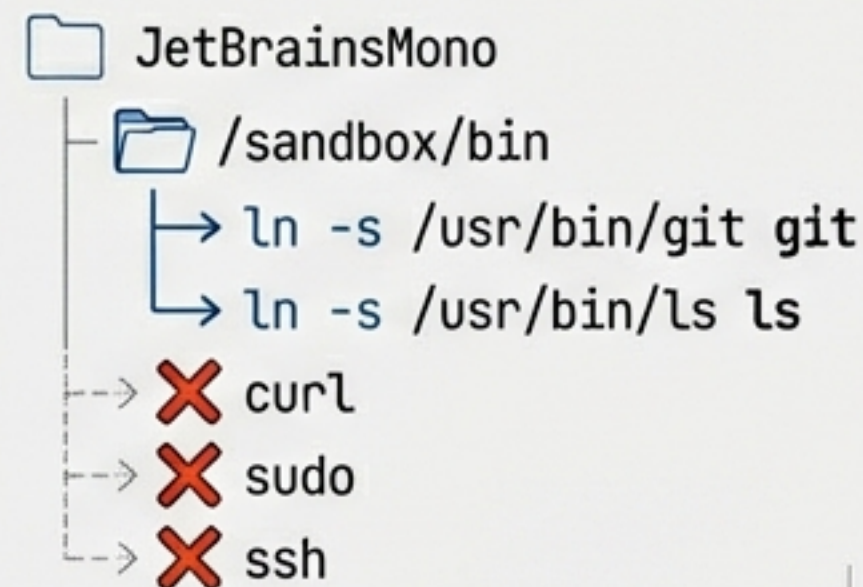
MECHANISM: THE RESTRICTED PATH

If the tool isn't there, it can't be used.



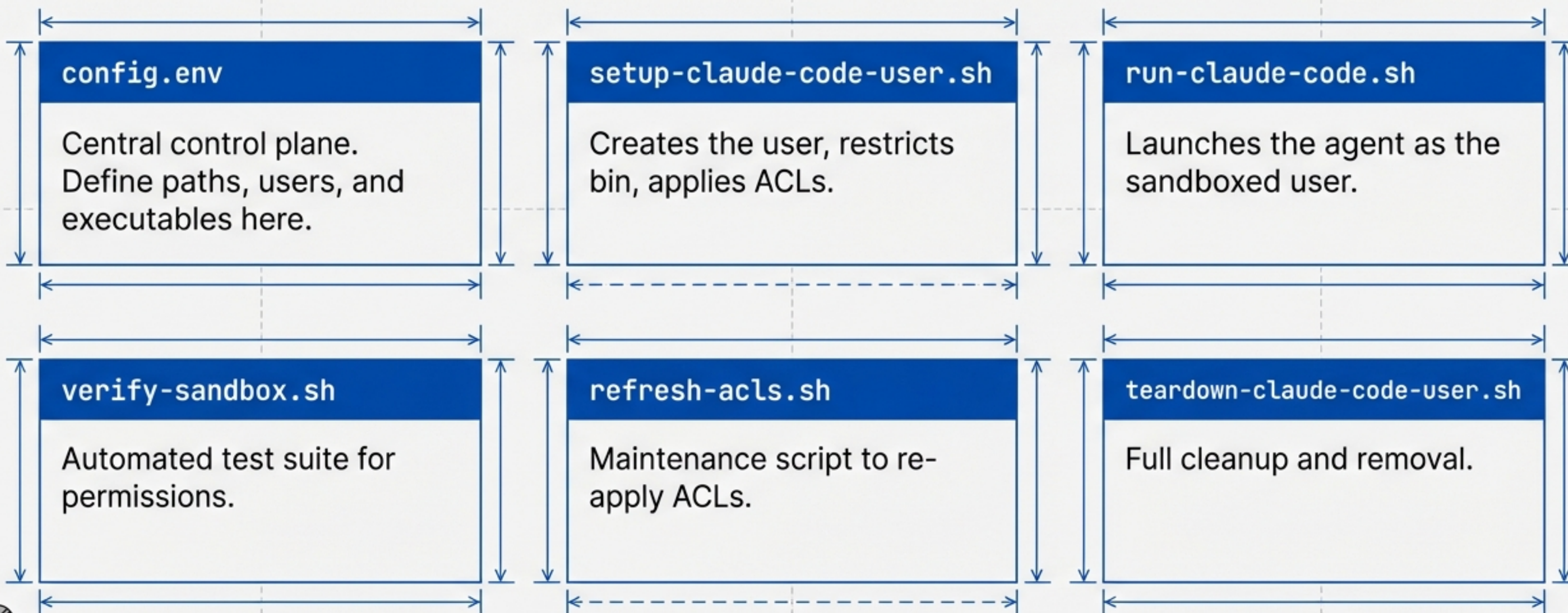
The sandbox uses a custom 'bin' directory containing only symlinks to approved tools.

It effectively neutralises malware downloaders and admin tools.



THE TOOLSET

A structured set of files for precision sandbox management.



STEP 1: CONFIGURATION (config.env)

CC_RW_DIRECTORIES: ○
Define Read/Write Projects here.

CC_RO_DIRECTORIES: ○
Define Read-Only libraries here.

CC_ALLOWED_EXECUTABLES: ○
List only necessary tools (git, node, npm).

```
# CC_USER="claude-code"
# CC_GROUP="claude-code"
# CC_HOME="/home/claude-code"
# CC_PASSWORD="change_me"

# Paths are relative to the host unless starting with /
# CC_RW_DIRECTORIES=(
#     "$HOME/my-project"
#     "/mnt/data/scratch"
# )

# CC_RO_DIRECTORIES=(
#     "/usr/include"
#     "/usr/share/doc"
#     "/usr/lib"
# )

# CC_ALLOWED_EXECUTABLES=(
#     "/usr/bin/git"
#     "/usr/bin/node"
#     "/usr/bin/npm"
#     "/usr/bin/env"
#     "/bin/ls"
#     "/bin/cp"
# )

# CC_EXTRA_APT_PACKAGES=()
```

⚠ **WARNING:** Change default CC_PASSWORD if interactive sudo is required. ⚠

STEP 2: DEPLOYMENT & VERIFICATION

DEPLOYMENT

Run the setup script to build the environment.

```
sudo ./setup-claude-code-user.sh
```

VERIFICATION

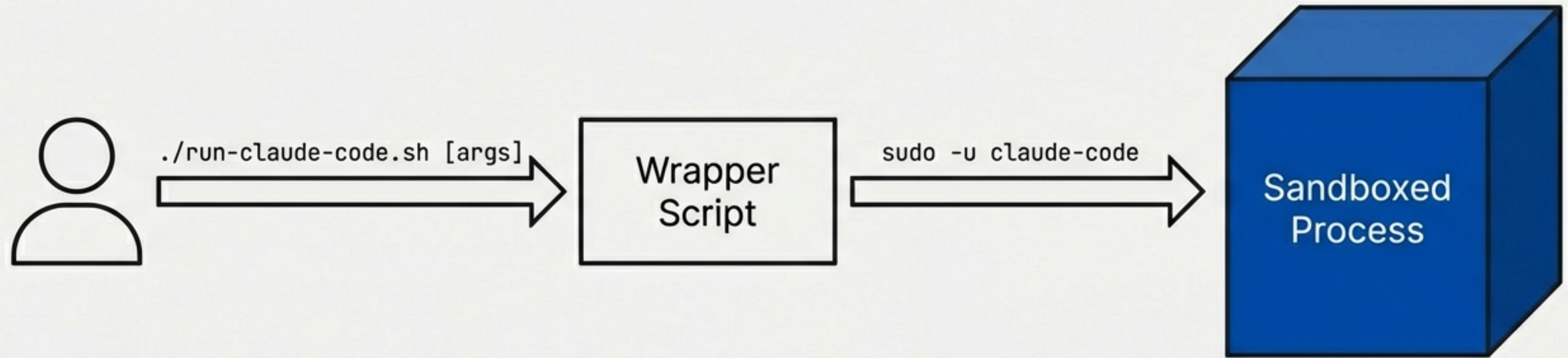
Run the test suite to prove isolation.

```
sudo ./verify-sandbox.sh
```

- ✓ Write access to Projects (Positive Test)
- ✗ Access to ~/.ssh (Negative Test - Blocked)
- ✗ Execution of curl (Negative Test - Blocked)

If verification fails, check `config.env` and run `refresh-acls.sh`.

WORKFLOW: RUNNING IN THE BOX

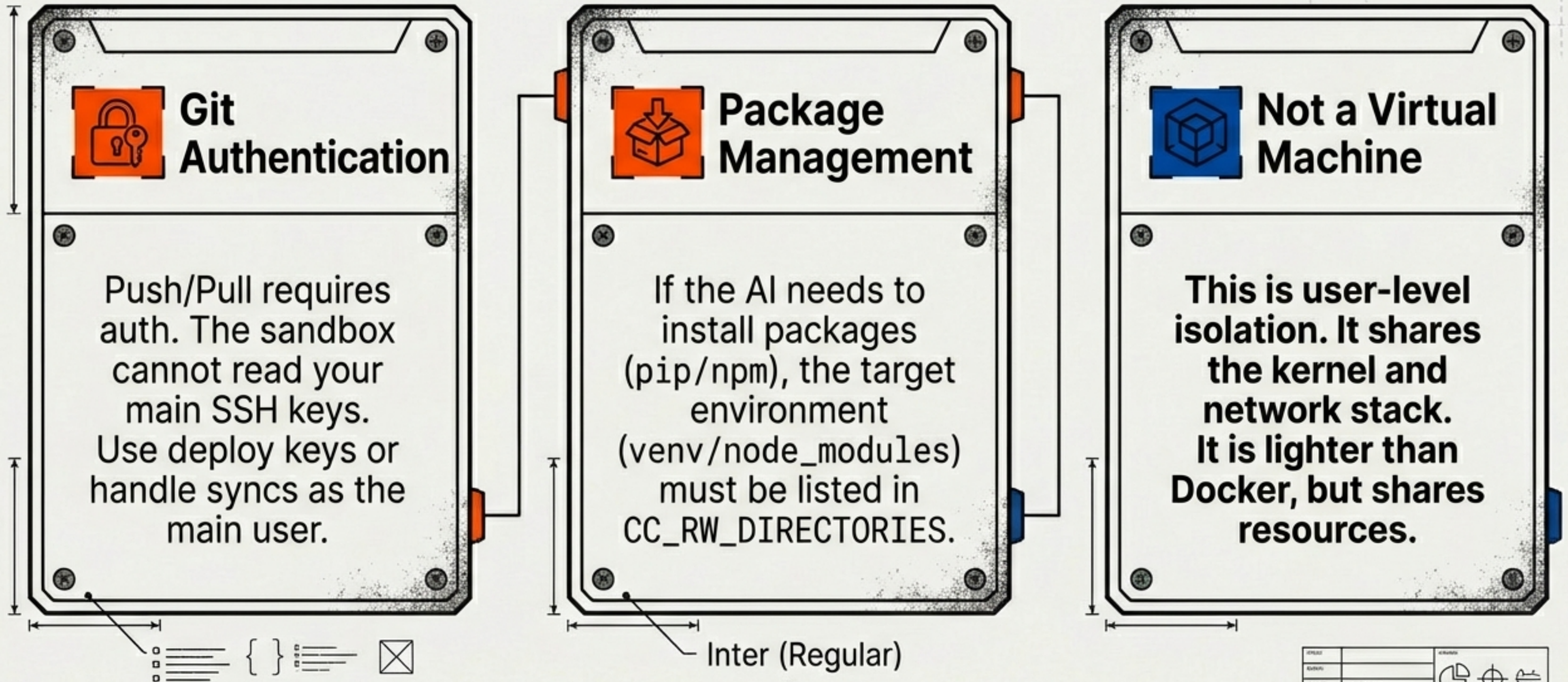


The wrapper script passes all arguments through to the underlying CLI. The experience feels native, but executes entirely as the isolated user. Shell history is segregated.

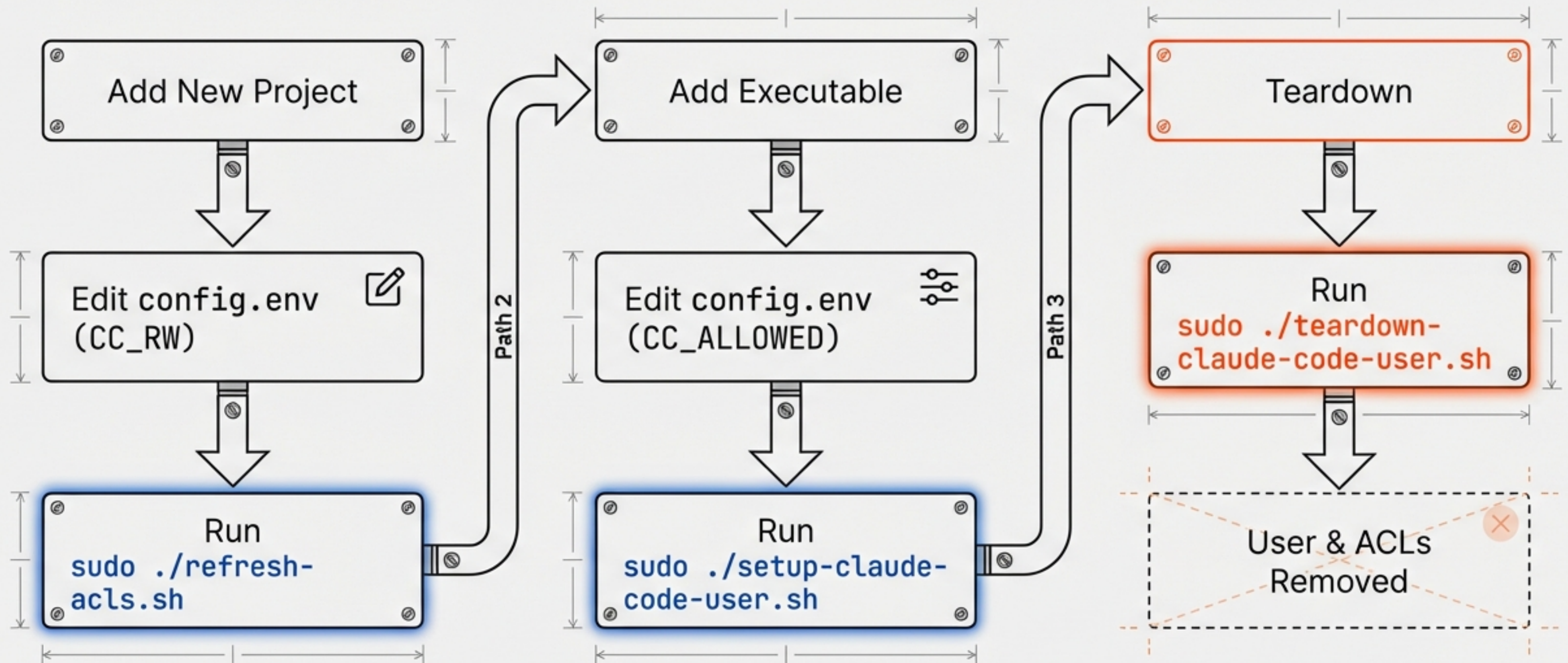
THREAT MODEL ANALYSIS

Threat	Default (Unsafe)	Sandboxed (Safe)
Exfiltration (Keys/Data)	Full access to ~/.ssh ~/.ssh in JetBrains Mono	BLOCKED: Permission Denied
Ransomware/Deletion Inter Regular	Can delete entire Home	CONTAINED: Only R/W dirs
System Integrity Inter Regular	Restricted by User	HARDENED: No sudo/admin
Malware Download Inter Regular	curl/wget available curl/wget in JetBrains Mono	BLOCKED: No network tools

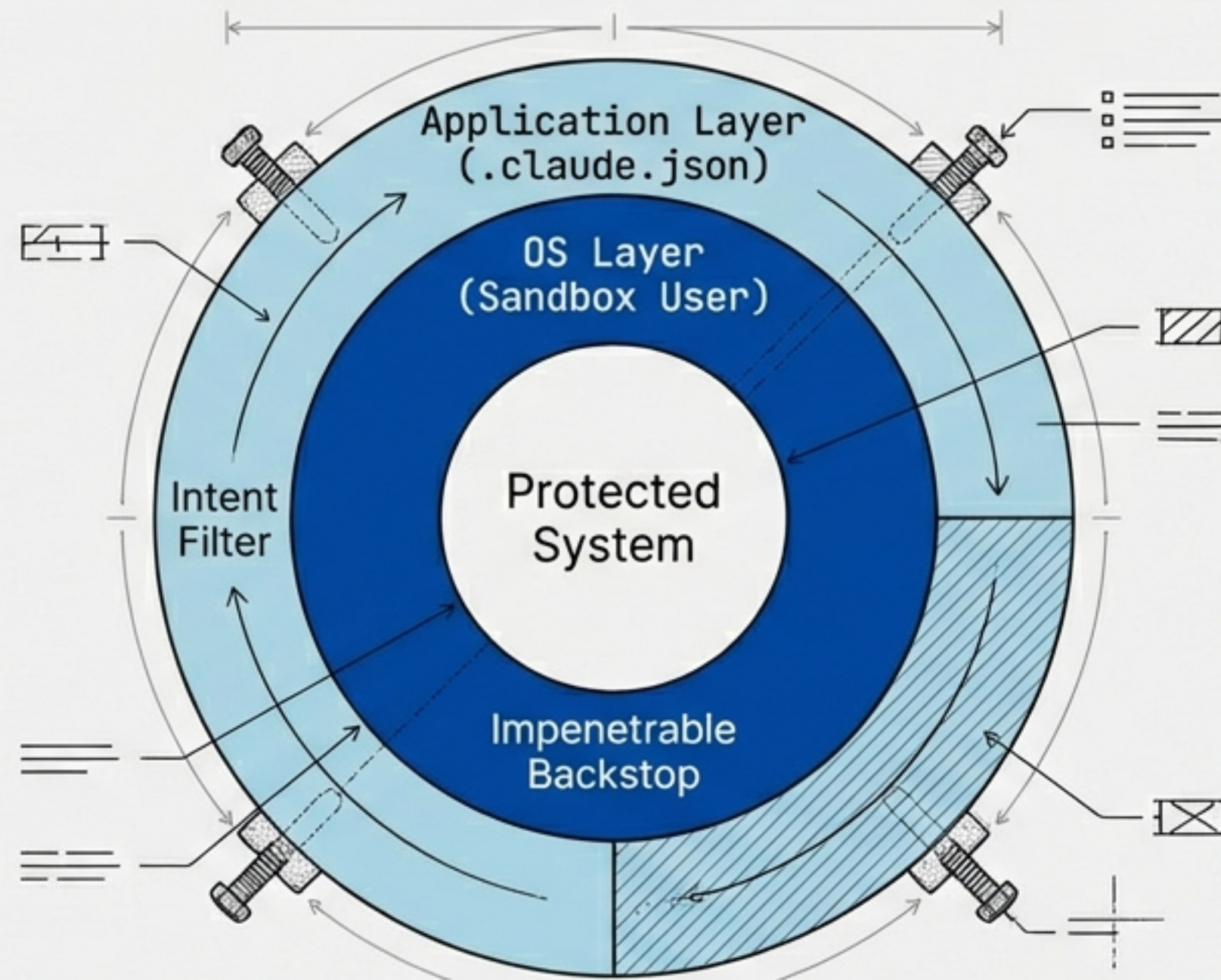
LIMITATIONS & CAVEATS



LIFECYCLE & MAINTENANCE



SUMMARY: DEFENSE IN DEPTH



The strongest posture combines application intent with OS constraints.

REVIEW CONFIG. ISOLATE.